运行 AI 交易引擎所需的 Python 库
使用方法: pip install -r requirements.txt
numpy>=1.20.0
pandas>=1.3.0
dataclasses; python_version < "3.7"
logging
如果需要连接真实 API，建议安装：
ib_insync (针对盈透证券)
yfinance (针对 Yahoo Finance 数据)

AI Bond & Rate Trading System - 本地部署指南

此项目是根据您的《Trading System All-In-One》方案书构建的 AI 驱动交易机器人原型。

文件说明

1. **TradingApp.jsx**: 前端指挥中心面板。
    - 采用 React + Tailwind CSS 构建。
    - 包含 Kill Switch、Ispread 监控、AI 信号流可视化。
2. **trading_engine.py**: 后端 AI 交易引擎。
    - 实现 WTI 与 Rates 的联动计算 (Ispread)。
    - 包含“黑天鹅”监测算法和多级风险状态机 (SAFE/WARNING/HALT)。
3. **requirements.txt**: Python 依赖清单。

如何在本地运行

1. 运行 AI 引擎 (Python)

确保您的电脑已安装 Python 3.8+，然后在终端执行：

```
pip install -r requirements.txt
python trading_engine.py
```

2. 运行 Dashboard (前端)

如果您想在本地运行前端界面：

1. 创建一个 React 项目：npx create-react-app my-trading-app
2. 安装必要插件：npm install lucide-react recharts
3. 将 TradingApp.jsx 的内容覆盖到项目的 src/App.js 中。
4. 运行：npm start

注意事项

- 当前代码中的数据为模拟 Feed。
- 如需实盘对接，请在 trading_engine.py 中修改 API 连接器部分 (如对接 IBKR 或 Bloomberg)。

```
import React, { useState, useEffect, useMemo } from 'react';
import {
```

```
  Activity, ShieldAlert, Zap, TrendingUp, AlertTriangle,
  Power, RefreshCcw, BarChart3, Layers, ShieldCheck,
  Bot, ZapOff, Globe, Database, Cpu, Lock, Unlock,
  ChevronRight, ArrowUpRight, ArrowDownRight, LineChart as LineChartIcon,
  Terminal, Shield
} from 'lucide-react';
import {
  LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
  ResponsiveContainer, AreaChart, Area, ComposedChart, Bar, Cell
} from 'recharts';

// 模拟金融数据生成逻辑
const generateMarketData = (count, base, volatility) => {
  return Array.from({ length: count }, (_, i) => ({
    time: i,
    rate: base + (Math.random() - 0.5) * volatility,
    wti: 75 + (Math.random() - 0.5) * 5,
    ispread: 12 + (Math.random() - 0.5) * 2,
    risk: Math.random() * 100
  }));
};

const App = () => {
  // 状态管理 - 严格遵循您的 PDF 架构 (SAFE/WARNING/EXIT_ONLY/HALT)
  const [systemStatus, setSystemStatus] = useState('SAFE');
  const [lifecycle, setLifecycle] = useState('PRE-LIVE'); // PRE-LIVE, GO-LIVE
  const [mode, setMode] = useState('NORMAL'); // NORMAL, REDUCE, RESTART
  const [isLocked, setIsLocked] = useState(true);

  // 信号流状态 (Feed -> Event -> Signal -> Approval -> Execution)
  const [signals, setSignals] = useState([]);
  const [executionLogs, setExecutionLogs] = useState([]);
  const [marketData, setMarketData] = useState(generateMarketData(40, 4.25,
0.5));

  // 实时数据模拟
  useEffect(() => {
    const interval = setInterval(() => {
      setMarketData(prev => {
        const last = prev[prev.length - 1];
        const nextRate = last.rate + (Math.random() - 0.5) * 0.05;
        const nextWti = last.wti + (Math.random() - 0.5) * 0.8;
        return [...prev.slice(1), {
          time: last.time + 1,
```

```
        rate: nextRate,
        wti: nextWti,
        ispread: (nextWti / nextRate) * 0.8,
        risk: Math.random() * 100
      }];
    });

    // 模拟自动生成交易信号 (如果处于 GO-LIVE 阶段)
    if (lifecycle === 'GO-LIVE' && Math.random() > 0.8 && systemStatus === 'SAFE') {
      const signalId = Math.random().toString(36).substr(2, 9);
      const signal = {
        id: signalId,
        type: Math.random() > 0.5 ? 'BOND_BUY' : 'RATE_SHORT',
        confidence: (0.8 + Math.random() * 0.18).toFixed(4),
        status: 'APPROVAL_PENDING',
        time: new Date().toLocaleTimeString()
      };
      setSignals(prev => [signal, ...prev].slice(0, 10));

      // 模拟执行流
      setTimeout(() => {
        setSignals(prev => prev.map(s => s.id === signalId ? { ...s, status: 'EXECUTED' } : s));
        setExecutionLogs(prev => [`[${new Date().toLocaleTimeString()}] 已执行 ${signal.type} @ 4.251% | AI CONF: ${signal.confidence}`, ...prev].slice(0, 15));
      }, 2000);
    }
  }, 3000);
  return () => clearInterval(interval);
 }, [lifecycle, systemStatus]);

 const handleKillSwitch = () => {
  setSystemStatus('HALT');
  setMode('REDUCE');
  setExecutionLogs(prev => [`[${new Date().toLocaleTimeString()}] !!! EMERGENCY HALT ENGAGED !!!`, ...prev]);
 };

 const statusConfig = {
  SAFE: { color: 'bg-emerald-500', text: 'text-emerald-400', border: 'border-emerald-500/30' },
  WARNING: { color: 'bg-amber-500', text: 'text-amber-400', border: 'border-amber-500/30' },
```

```
    EXIT_ONLY: { color: 'bg-orange-600', text: 'text-orange-400', border: 'border-orange-500/30' },
    HALT: { color: 'bg-red-600', text: 'text-red-500', border: 'border-red-500/30' }
  };

  return (
    <div className="flex flex-col h-screen bg-[#020408] text-slate-300 font-mono overflow-hidden">
      {/* 顶部导航与全局控制 */}
      <nav className="h-14 border-b border-white/5 bg-black/40 backdrop-blur-xl flex items-center justify-between px-6 shrink-0">
        <div className="flex items-center gap-3">
          <div className="w-8 h-8 bg-indigo-600 rounded flex items-center justify-center shadow-lg shadow-indigo-500/20">
            <Bot size={18} className="text-white" />
          </div>
          <div>
            <h1 className="text-xs font-black tracking-[0.2em] text-white uppercase italic">Trading Command Center</h1>
            <div className="flex gap-3 text-[9px] opacity-60">
              <span className="flex items-center gap-1"><Database size={10} /> FEED: LIVE</span>
              <span className="flex items-center gap-1"><Cpu size={10} /> MODEL: V4.2</span>
            </div>
          </div>
        </div>

        <div className="flex items-center gap-4">
          <div className="flex gap-1">
            {Object.keys(statusConfig).map(s => (
              <div key={s} className={`w-1.5 h-1.5 rounded-full ${systemStatus === s ? statusConfig[s].color : 'bg-white/10'}`} />
            ))}
          </div>

          <div className={`px-3 py-1 rounded text-[10px] font-bold border transition-all ${lifecycle === 'GO-LIVE' ? 'bg-emerald-500/10 border-emerald-500/50 text-emerald-400' : 'bg-slate-800 border-slate-700 text-slate-500'}`}>
            {lifecycle}
          </div>

          <button
```

```
          onClick={() => setIsLocked(!isLocked)}
          className={`p-1.5 rounded hover:bg-white/5 transition-colors ${!
isLocked ? 'text-amber-500' : ''}`}
        >
          {isLocked ? <Lock size={16} /> : <Unlock size={16} />}
        </button>

        <button
          disabled={isLocked}
          onClick={handleKillSwitch}
          className={`flex items-center gap-2 px-4 py-1.5 rounded text-[10px]
font-bold transition-all ${systemStatus === 'HALT' ? 'bg-red-600 text-white' :
'bg-red-950/20 text-red-500 border border-red-900/50 hover:bg-red-600
hover:text-white'}`}
        >
          <ZapOff size={14} /> KILL SWITCH
        </button>
      </div>
    </nav>

    {/* 主工作区 */}
    <main className="flex-1 overflow-hidden p-4 grid grid-cols-12 gap-4">

      {/* 左侧：资产监控与 Ispread */}
      <div className="col-span-12 lg:col-span-8 flex flex-col gap-4 overflow-y-
auto pr-1">

        {/* 四大核心状态卡片 */}
        <div className="grid grid-cols-4 gap-4">
          <div className={`p-4 rounded-xl border $
{statusConfig[systemStatus].border} bg-white/[0.02] flex flex-col justify-
between`}>
            <span className="text-[10px] font-bold text-slate-500 uppercase
tracking-widest">Global Status</span>
            <span className={`text-xl font-black italic $
{statusConfig[systemStatus].text}`}>{systemStatus}</span>
          </div>
          <div className="p-4 rounded-xl border border-white/5 bg-white/[0.02]
flex flex-col justify-between">
            <span className="text-[10px] font-bold text-slate-500 uppercase
tracking-widest">Mode</span>
            <span className="text-xl font-black italic text-white">{mode}</span>
          </div>
          <div className="p-4 rounded-xl border border-white/5 bg-white/[0.02]
```

```
flex flex-col justify-between">
            <span className="text-[10px] font-bold text-slate-500 uppercase
tracking-widest">WTI Spot</span>
            <span className="text-xl font-black italic text-blue-400">$
{marketData[marketData.length-1].wti.toFixed(2)}</span>
          </div>
          <div className="p-4 rounded-xl border border-white/5 bg-white/[0.02]
flex flex-col justify-between">
            <span className="text-[10px] font-bold text-slate-500 uppercase
tracking-widest">10Y Yield</span>
            <span className="text-xl font-black italic text-
emerald-400">{marketData[marketData.length-1].rate.toFixed(3)}%</span>
          </div>
        </div>

        {/* Ispread & Rates 联动监控 */}
        <div className="flex-1 bg-white/[0.02] border border-white/5 rounded-2xl
p-6 relative min-h-[350px]">
          <div className="flex justify-between items-center mb-6">
            <h2 className="text-xs font-bold flex items-center gap-2 tracking-
widest">
              <LineChartIcon size={14} className="text-indigo-500" /> ISPREAD
ANALYSIS (WTI VS BOND)
            </h2>
            <div className="flex gap-4 text-[10px]">
              <span className="flex items-center gap-1"><div className="w-2 h-2
bg-indigo-500 rounded-full" /> WTI PRICE</span>
              <span className="flex items-center gap-1"><div className="w-2 h-2
bg-emerald-500 rounded-full" /> BOND RATE</span>
            </div>
          </div>
          <div className="h-64">
            <ResponsiveContainer width="100%" height="100%">
              <ComposedChart data={marketData}>
                <CartesianGrid strokeDasharray="3 3" stroke="#ffffff05"
vertical={false} />
                <XAxis dataKey="time" hide />
                <YAxis yAxisId="left" hide domain={['auto', 'auto']} />
                <YAxis yAxisId="right" orientation="right" hide domain={['auto',
'auto']} />
                <Tooltip
                  contentStyle={{ backgroundColor: '#000', border: '1px solid #ffffff10',
fontSize: '10px' }}
                />
```

```
              <Area yAxisId="left" type="monotone" dataKey="wti" fill="#6366f1"
fillOpacity={0.05} stroke="#6366f1" strokeWidth={2} isAnimationActive={false} />
              <Line yAxisId="right" type="monotone" dataKey="rate"
stroke="#10b981" strokeWidth={2} dot={false} isAnimationActive={false} />
              <Bar yAxisId="left" dataKey="ispread" fill="#ffffff" opacity={0.05}
barSize={2} />
            </ComposedChart>
          </ResponsiveContainer>
        </div>
      </div>

      {/* 执行日志 */}
      <div className="h-48 bg-black/60 border border-white/5 rounded-2xl p-4
overflow-hidden flex flex-col">
        <h3 className="text-[10px] font-bold text-slate-500 mb-2 uppercase flex
items-center gap-2">
          <Terminal size={12} /> Execution Analytics & Logs
        </h3>
        <div className="flex-1 overflow-y-auto space-y-1 pr-2">
          {executionLogs.map((log, i) => (
            <div key={i} className="text-[9px] text-slate-400 border-l border-
slate-800 pl-3 leading-relaxed">
              <span className="text-indigo-500 mr-2">►</span> {log}
            </div>
          ))}
          {executionLogs.length === 0 && <div className="text-[9px] opacity-20
italic">Awaiting automated execution events...</div>}
        </div>
      </div>
    </div>

    {/* 右侧：决策与风险控制 */}
    <div className="col-span-12 lg:col-span-4 flex flex-col gap-4">

      {/* 生命周期控制 */}
      <div className="bg-indigo-500/5 border border-indigo-500/20
rounded-2xl p-4">
        <div className="flex justify-between items-center mb-3">
          <span className="text-[10px] font-bold text-indigo-400 uppercase
tracking-tighter">System Lifecycle</span>
          <button
            disabled={isLocked}
            onClick={() => setLifecycle(lifecycle === 'PRE-LIVE' ? 'GO-LIVE' : 'PRE-
LIVE')}
```

```
            className={`px-2 py-1 rounded text-[9px] font-bold uppercase
transition-all ${lifecycle === 'GO-LIVE' ? 'bg-emerald-600 text-white' : 'bg-
slate-800 text-slate-500'}`}
           >
            Switch Phase
           </button>
          </div>
          <div className="grid grid-cols-2 gap-2">
           <div className={`h-1.5 rounded-full ${lifecycle === 'PRE-LIVE' ? 'bg-
indigo-500 shadow-[0_0_8px_#6366f1]' : 'bg-white/5'}`} />
           <div className={`h-1.5 rounded-full ${lifecycle === 'GO-LIVE' ? 'bg-
emerald-500 shadow-[0_0_8px_#10b981]' : 'bg-white/5'}`} />
          </div>
         </div>

         {/* 实时信号队列 */}
         <div className="flex-1 bg-white/[0.02] border border-white/5 rounded-2xl
p-4 flex flex-col overflow-hidden">
          <h3 className="text-[10px] font-bold text-slate-500 mb-4 flex items-
center gap-2 uppercase">
           <Zap size={14} className="text-amber-500" /> AI Decision Queue
          </h3>
          <div className="flex-1 overflow-y-auto space-y-2 pr-1">
           {signals.map(s => (
            <div key={s.id} className="p-3 bg-white/[0.03] border border-white/5
rounded-xl hover:border-white/10 transition-all group">
             <div className="flex justify-between items-center mb-2">
              <span className={`text-[9px] px-1.5 py-0.5 rounded font-black $
{s.type.includes('BUY') ? 'bg-blue-500/20 text-blue-400' : 'bg-purple-500/20
text-purple-400'}`}>
               {s.type}
              </span>
              <span className="text-[9px] opacity-40">{s.time}</span>
             </div>
             <div className="flex justify-between items-end">
              <div>
               <div className="text-[8px] text-slate-500 uppercase">AI
Confidence</div>
               <div className="text-sm font-bold text-white tracking-
tighter">{s.confidence}</div>
              </div>
              <div className="text-right">
               <span className={`text-[8px] font-bold uppercase ${s×status ===
'EXECUTED' ? 'text-emerald-400' : 'text-amber-500 animate-pulse'}`}>
```

```
                {s.status}
              </span>
            </div>
          </div>
        </div>
      ))}
      {signals.length === 0 && (
        <div className="h-full flex flex-col items-center justify-center
opacity-10 italic text-[10px]">
          <Globe size={30} className="mb-2" />
          Waiting for Signals...
        </div>
      )}
    </div>
  </div>

  {/* 风险守护 (Crash/Spike/Black Swan) */}
  <div className="bg-red-500/5 border border-red-500/20 rounded-2xl
p-4">
    <h3 className="text-[10px] font-bold text-red-500 mb-4 flex items-
center gap-2 uppercase tracking-widest">
      <Shield size={14} /> Risk Guard Engine
    </h3>
    <div className="space-y-4">
      <div>
        <div className="flex justify-between text-[9px] mb-1.5">
          <span className="text-slate-500">BLACK SWAN PROBABILITY</
span>
          <span className="text-white font-bold">1.2%</span>
        </div>
        <div className="w-full h-1 bg-white/5 rounded-full overflow-hidden">
          <div className="h-full bg-red-500 w-[15%]" />
        </div>
      </div>
      <div className="grid grid-cols-2 gap-2">
        <button
          disabled={isLocked}
          onClick={() => setMode('REDUCE')}
          className="py-2 bg-orange-950/20 border border-orange-900/40
text-orange-500 text-[9px] font-bold rounded-lg uppercase tracking-tighter"
        >
          Reduce Mode
        </button>
        <button
```

```
                disabled={isLocked}
                onClick={() => setSystemStatus('SAFE')}
                className="py-2 bg-emerald-950/20 border border-emerald-900/40
text-emerald-500 text-[9px] font-bold rounded-lg uppercase tracking-tighter"
              >
                Clear Alert
              </button>
            </div>
          </div>
        </div>

      </div>
    </main>

    {/* 底部状态条 */}
    <footer className="h-6 bg-black border-t border-white/5 flex items-center
px-4 justify-between shrink-0">
      <div className="flex gap-4">
        <div className="flex items-center gap-1.5">
          <div className="w-1.5 h-1.5 rounded-full bg-emerald-500 shadow-
[0_0_5px_#10b981]" />
          <span className="text-[9px] font-bold text-slate-500
uppercase">System Integrity: Nominal</span>
        </div>
        <div className="text-[9px] text-slate-600 truncate max-w-md">
          CORE: Processing WTI-BOND spread parity ... No spikes detected ...
Latency: 14ms
        </div>
      </div>
      <div className="text-[9px] text-slate-500 flex items-center gap-2">
        {new Date().toISOString()}
      </div>
    </footer>
  </div>
 );
};

export default App;
import time
import random
import logging
from dataclasses import dataclass
from enum import Enum
```

```
# 1. 系统状态与枚举 (严格遵循您的 PDF 方案书)
class SystemStatus(Enum):
    SAFE = "SAFE"
    WARNING = "WARNING"
    EXIT_ONLY = "EXIT_ONLY"
    HALT = "HALT"

class Lifecycle(Enum):
    PRE_LIVE = "PRE-LIVE"
    GO_LIVE = "GO-LIVE"

@dataclass
class MarketData:
    timestamp: float
    wti_price: float
    bond_yield: float
    ispread: float

# 2. 核心 AI 交易引擎类
class BondTradingAI:
    def __init__(self):
        self.status = SystemStatus.SAFE
        self.lifecycle = Lifecycle.PRE_LIVE
        self.kill_switch_active = False

        # 风险阈值
        self.vol_threshold = 0.05  # 5% 波动进入 Warning
        self.crash_threshold = 0.12 # 12% 波动进入 Halt (Black Swan)

        logging.basicConfig(level=logging.INFO, format='%(asctime)s - [%
(levelname)s] - %(message)s')
        self.logger = logging.getLogger("AIBondEngine")

    # ------------------------------------------------------------
    # 核心算法: Ispread 计算
    # ------------------------------------------------------------
    def calculate_ispread(self, wti, bond_rate):
        """
        计算原油 (WTI) 与利率 (Rates) 的联动价差。
        当 Ispread 偏离历史均值时，AI 模型会捕捉套利信号。
        """
        if bond_rate == 0: return 0
        return (wti / bond_rate) * 0.85
```

```
    # ------------------------------------------------------------
    # 风险扫描: Black Swan Guard
    # ------------------------------------------------------------
    def scan_risk(self, current, previous):
        if not previous: return

        # 检查利率的瞬间脉冲波动 (Spike Detection)
        rate_change = abs(current.bond_yield - previous.bond_yield) /
previous.bond_yield

        if rate_change > self.crash_threshold:
            self.trigger_kill_switch(f"BLACK SWAN DETECTED: Rate moved
{rate_change:.2%}")
        elif rate_change > self.vol_threshold:
            self.status = SystemStatus.WARNING
            self.logger.warning(f"SYSTEM WARNING: High Volatility
({rate_change:.2%}) - Entering Reduce Mode")
        else:
            if self.status != SystemStatus.HALT:
                self.status = SystemStatus.SAFE

    def trigger_kill_switch(self, reason):
        self.kill_switch_active = True
        self.status = SystemStatus.HALT
        self.logger.critical(f"!!! KILL SWITCH ACTIVATED !!! REASON: {reason}")
        self.logger.info("Executing EXIT_ONLY sequence: Closing all active
positions.")

    # ------------------------------------------------------------
    # AI 决策逻辑
    # ------------------------------------------------------------
    def process_ai_signal(self, data):
        """
        AI 信号发生器入口。
        """
        confidence = random.uniform(0.6, 0.99)

        # 简单的均值回归模拟逻辑
        if data.ispread > 15.0:
            return {"action": "SELL_BOND", "confidence": confidence}
        elif data.ispread < 10.0:
            return {"action": "BUY_BOND", "confidence": confidence}
```

```
        return None

    # ------------------------------------------------------------
    # 模拟运行
    # ------------------------------------------------------------
    def run_simulation(self):
        self.logger.info(f"Engine Initialized. Phase: {self.lifecycle.value}")
        last_data = None

        # 模拟 100 个周期的数据流
        for i in range(100):
            if self.kill_switch_active: break

            # 模拟获取实时 Feed
            curr_wti = 75.0 + random.uniform(-1, 1)
            curr_rate = 4.25 + random.uniform(-0.02, 0.02)
            ispread = self.calculate_ispread(curr_wti, curr_rate)

            current_data = MarketData(time.time(), curr_wti, curr_rate, ispread)

            # 风险扫描
            self.scan_risk(current_data, last_data)

            # 信号处理
            if self.lifecycle == Lifecycle.GO_LIVE and self.status ==
SystemStatus.SAFE:
                signal = self.process_ai_signal(current_data)
                if signal and signal['confidence'] > 0.85:
                    self.logger.info(f"AI EXECUTION: {signal['action']} (Confidence:
{signal['confidence']:.4f})")

            last_data = current_data
            time.sleep(1) # 每秒更新

if __name__ == "__main__":
    ai_bot = BondTradingAI()
    ai_bot×lifecycle = Lifecycle.GO_LIVE
    # ai_bot.run_simulation() # 本地运行时取消此行注释
```